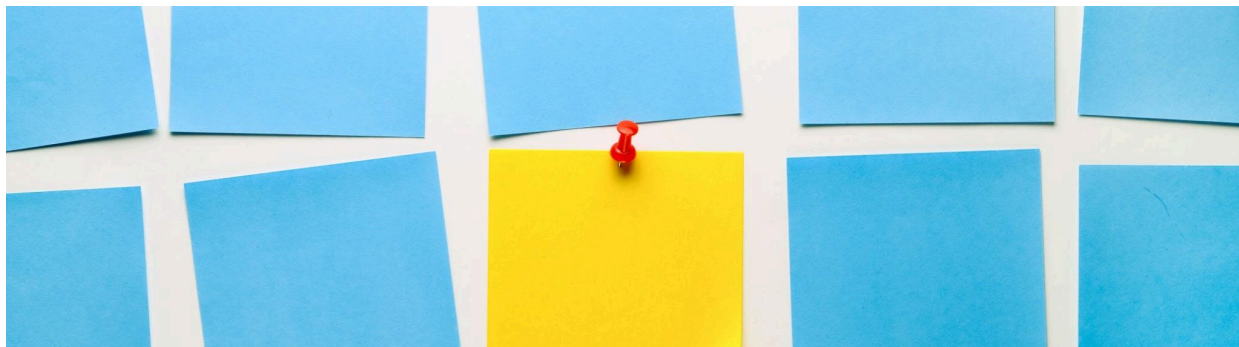




الگوریتم ساخت واژگان (آموزش WordPiece)

را منتشر نکرده است، لذا بسیاری از پیاده‌سازی‌ها بر اساس WordPiece نکته مهم: گوگل نسخه اصلی آموزش حدس‌ها و ترجمه‌ی ادبیات هستند.

مراحل کلی به این صورت‌اند:

1. شروع با واژگان اولیه

- ابتدا واژگان اولیه شامل همه کاراکترهایی است که در متن دیده شده‌اند (به علاوه نشانه‌های خاص مثل [SEP]، [CLS]، [UNK] و غیره).
- همچنین در مدل‌هایی مانند BERT، از پیشوندی مانند ## برای نشان دادن اینکه یک قطعه زیرواژه‌ای است که ادامه‌ی یک واژه‌ی قبلی است، استفاده می‌کنند.

2. شکستن واژه‌ها به قطعات اولیه

- هر واژه در داده آموزش به قطعات اولیه تقسیم می‌شود؛ معمولاً به کاراکترها همراه پیشوند ## برای کاراکترهایی که در میانه واژه‌اند.
- مثلاً واژه‌ی "word" ممکن است به ["w", "##o", "##r", "##d"] تقسیم شود.

3. محاسبه فراوانی‌ها

- برای هر قطعه زیرواژه‌ای فعلی و هر جفت مجاور (pair) از قطعات، تعداد وقوع آن‌ها را در کل داده محاسبه می‌کنند.
- همچنین باید بدانند در چه موضعی قطعه در ابتدای واژه است یا در وسط واژه (به علت پیشوند ##).

4. ادغام‌های تدریجی (merging)

- تا زمانی که به اندازهٔ واژگان مورد نظر برسند یا دیگر بهبود معنی‌داری وجود نداشته باشد، عملیات ادغام ادامه می‌یابد.
- در هر تکرار، یک جفت قطعه (A, B) انتخاب می‌شود که اگر آن‌ها را با هم ادغام کنیم (یعنی قطعه جدید AB بسازیم) باعث بیشترین «افزایش احتمال مدل» شود.
- برخلاف BPE که فقط معمولاً جفتی با بیشترین فراوانی را ادغام می‌کند، WordPiece از معیاری مبتنی بر احتمال استفاده می‌کند تا میزان بهبود مدل را در نظر بگیرد.

■ به طور خاص، معیار یا امتیاز برای یک جفت (A, B) می‌تواند چیزی مانند:

$$((\text{score}(A,B)=P(AB))/(P(A) P(B))$$

یعنی فراوانی مشترک تقسیم بر حاصل ضرب فراوانی‌های تک‌تک قطعات.

■ جفتی که این امتیاز را بیشینه می‌کند انتخاب می‌شود و ادغام صورت می‌گیرد.

- بعد از ادغام، فراوانی‌ها باید به‌روز شوند و تکرار ادامه یابد.
- در نهایت ما به یک واژگان نهایی شامل قطعات (که بعضی از آن‌ها ممکن است ترکیبی از کاراکترها باشند) دست پیدا می‌کنیم.

به این ترتیب، مدل به جای اینکه تمام واژه‌ها را به‌صورت کامل ببیند، به تدریج قطعات معنادار را یاد می‌گیرد.

الگوریتم رمزگذاری (Tokenization) با واژگان آموزش دیده شده

بعد از اینکه واژگان قطعات (subwords) ساخته شد، وقتی بخوایم یک متن جدید را توکن کنیم، الگوریتم به این صورت عمل می‌کند:

1. متن را ابتدا به کلمات پایه تقسیم می‌کنند (با شکستن فواصل، نقطه‌گذاری، جداسازی واژه‌ها).

2. سپس برای هر واژه، از متد طولانی‌ترین تطابق (*longest-match-first*) یا ماکزیمم تطبیق (*MaxMatch*) استفاده می‌شود:

- از ابتدای واژه شروع می‌کنند و سعی می‌کنند بزرگ‌ترین قطعه‌ای که در واژگان وجود دارد را تطبیق دهند.
- اگر آن قطعه پیدا شد، آن را به عنوان یک توکن انتخاب می‌کنند و بقیه واژه (باقیمانده کاراکترها) را به همین روش پردازش می‌کنند.
- اگر در میانه واژه باشند، قطعه باید با پیشوند **##** شروع شود تا نشان دهد ادامه یک واژه است.
- اگر هیچ قطعه‌ای نتوان پیدا کرد (یعنی بخشی از واژه در واژگان نیست)، ممکن است کل واژه به **[UNK]** (توکن ناشناخته) تبدیل شود یا بخش‌های کوچکتر به کاراکتر تقسیم شوند (بسته به پیاده‌سازی).

مثال ساده: واژه‌ی "hugs"

- فرض کن واژگان شامل **["hug", "##s"]** باشد (و قطعات کاراکتری پایه). پس وقتی **"hugs"** را بخوایم توکن کنیم:

- ابتدا بزرگ‌ترین قطعه‌ای که در ابتدای واژه وجود دارد: **"hug"** → تطبیق دارد.
- باقی مانده **"s"** (در موقعیت میانه) باید به صورت **s##** تطبیق داده شود (اگر در واژگان باشد).
- نتیجه توکن‌ها: **["hug", "##s"]**

اگر واژه‌ای مانند "bugs" باشد و واژگان شامل قطعاتی مثل ["b", "##u", "##gs"] باشد، ممکن است ساخته شود: ["b", "##u", "##gs"]. اگر در میانه واژه نتوان تطبیق داد، ممکن است کل واژه به [UNK] تبدیل شود.

در مقاله «Fast WordPiece Tokenization»، گوگل روشی پیشنهاد کرده است که پیچیدگی زمانی رمزگذاری را از حالت نمایی یا تقریباً $O(n^2)$ به حالت خطی $O(n)$ کاهش می‌دهد، با استفاده از ساختار داده‌ی Trie و برخی پیوندهای شکست (failure links) مشابه الگوریتم Aho-Corasick. به این صورت در هنگام جستجوی قطعات واژگان در واژه‌ورودی، وقتی شاخه‌ای از trie نتواند ادامه یابد، به جای برگشت کامل، از لینک شکست استفاده می‌شود تا سریع‌تر شاخه مناسب بعدی را پیدا کند.

مقایسه WordPiece با BPE و سایر روش‌ها

چند نکته متمایز کننده:

- در BPE، معمولاً جفتی از نمادها که بیشترین فراوانی را دارند ادغام می‌شوند. اما در WordPiece، انتخاب جفت ادغام بر مبنای امتیازی مبتنی بر احتمال / سوددهی به مدل است.
- در توکن کردن واژه جدید، WordPiece از روش طولانی‌ترین تطابق استفاده می‌کند؛ اما در BPE ممکن است از اعمال ترتیبی قوانین ادغام استفاده شود.
- در استفاده عملی، WordPiece برای مدل‌هایی مانند BERT استفاده شده است.
- نکته بهینه‌سازی: گوگل با روش Fast WordPiece توانسته است فرآیند tokenization را بسیار سریع‌تر کند (تا 8 برابر سریع‌تر نسبت به پیاده‌سازی‌های معمول).

کد نمونه پیاده سازی این روش:

```
from collections import defaultdict, Counter
from typing import List, Dict, Tuple
```

```
class WordPieceTokenizer:
    def __init__(self, vocab_size: int = 1000, unk_token: str = "[UNK]"):

```

```

self.vocab_size = vocab_size
self.unk_token = unk_token
self.vocab: Dict[str, int] = {}

def _init_vocab(self, corpus: List[str]):
    """Initialize vocab with individual characters."""
    char_counter = Counter()
    for word in corpus:
        for ch in word:
            char_counter[ch] += 1
    # start with unique characters
    self.vocab = {ch: freq for ch, freq in char_counter.items()}

def _get_stats(self, corpus: List[List[str]]) -> Dict[Tuple[str, str], int]:
    """Count frequency of pairs of tokens in corpus."""
    pairs = defaultdict(int)
    for word in corpus:
        for i in range(len(word) - 1):
            pairs[(word[i], word[i + 1])] += 1
    return pairs

def _merge_vocab(self, pair: Tuple[str, str], corpus: List[List[str]]) -> List[List[str]]:
    """Merge the most frequent pair into a single token."""
    new_corpus = []
    bigram = " ".join(pair)
    replacement = "".join(pair)
    for word in corpus:
        new_word = []
        i = 0
        while i < len(word):
            if i < len(word) - 1 and (word[i], word[i+1]) == pair:
                new_word.append(replacement)
                i += 2
            else:
                new_word.append(word[i])
                i += 1
        new_corpus.append(new_word)
    return new_corpus

def train(self, texts: List[str]):
    """Train tokenizer on a list of words."""
    # split texts into words and then into characters
    corpus = [[ch for ch in word] for word in texts]
    self._init_vocab(texts)

```

```

while len(self.vocab) < self.vocab_size:
    pairs = self._get_stats(corpus)
    if not pairs:
        break
    best_pair = max(pairs, key=pairs.get)
    corpus = self._merge_vocab(best_pair, corpus)
    new_token = "".join(best_pair)
    self.vocab[new_token] = pairs[best_pair]

def tokenize(self, word: str) -> List[str]:
    """Tokenize a new word using the trained vocab (greedy longest-match)."""
    if not self.vocab:
        raise ValueError("Tokenizer is not trained yet.")

    tokens = []
    i = 0
    while i < len(word):
        match = None
        # longest match search
        for j in range(len(word), i, -1):
            sub = word[i:j]
            if sub in self.vocab:
                match = sub
                break
        if match is None:
            tokens.append(self.unk_token)
            i += 1
        else:
            tokens.append(match)
            i += len(match)
    return tokens

if __name__ == "__main__":
    corpus = ["playing", "plays", "played", "player", "playful"]
    tokenizer = WordPieceTokenizer(vocab_size=50)
    tokenizer.train(corpus)

    test_word = "playground"
    print("Input:", test_word)
    print("Tokens:", tokenizer.tokenize(test_word))

```

پیاده سازی به کمک لایبرری‌ها:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Unbelievable happiness!"
tokens = tokenizer.tokenize(text)
ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens:", tokens)
print("IDs:", ids)
```

منابع

<https://research.google/blog/a-fast-wordpiece-tokenization-system/>

به کمک AI GPT